Inryeol Choi

Backend Portfolio

A project record focused on backend systems that connect service flow, data structure, authentication, and runtime environments.

dlsfuf0316@gmail.com | github.com/InryeolChoi | Seoul, South Korea

Portfolio Focus

- Treat APIs as structured user flows that connect data state, validation, exceptions, and operational behavior.
- Work across foundational backend concerns such as authentication, databases, deployment, and runtime configuration.
- Built system programming fundamentals through hands-on projects involving networking, processes, file descriptors, and concurrency.

Award

Daytona HackSprint Seoul 2026 - 2nd Place (participant voting) | AgentTrap | AI Builders | October 18, 2026 | Two developers, about two hours

Representative Projects

AgentTrap

AI Agent Security / Prompt Injection | github.com/InryeolChoi/agenttrap

An interactive playground where users inject instructions into an email to test an AI Defender's security and its ability to complete the original summary task.

Problem

If the Defender ignores an attack but fails its task, should that count as success? How can reading malicious text be distinguished from following it?

Approach

A Python judge compares a normal baseline with the attack and checks unauthorized tool-call attempts separately from required summary facts. Daytona provides isolated execution and logs; Nosana provides Defender inference.

Result

A failed baseline makes the attack inconclusive, and blocked tool calls are recorded as attempts rather than executions. Built by a two-person team in approximately two hours.

- Designed separate security and task-completion criteria.
- Used a fixed email-summary scenario with a Streamlit input and results UI.
- Connected Daytona sandbox execution with Nosana inference.

Tech Stack

Python	Daytona	Nosana	Streamlit

ft_malloc

Thread-safe dynamic memory allocator in C

Implemented malloc, free, realloc, show_alloc_mem, and show_alloc_mem_ex on top of mmap, then extended the allocator for concurrent access.

Problem

A memory allocator must handle alignment, fragmentation, invalid pointers, size overflow, and munmap failure while remaining consistent under concurrent access.

Approach

Tracked allocation with boxes and tag metadata across TINY, SMALL, and LARGE zones. Implemented free-tag reuse, splitting, and coalescing, protected each zone with a pthread mutex, and updated box lists only after munmap succeeds.

Result

Achieved 16-byte alignment and failure-safe release, and verified multithreaded stress runs without deadlocks or memory corruption.

- Implemented malloc, free, realloc, and memory inspection functions.
- Added free-tag lookup, splitting, reuse, and adjacent-tag coalescing.
- Printed live user areas as hexadecimal dumps in show_alloc_mem_ex.

Tech Stack

C	mmap	pthread mutex	Dynamic memory
Thread safety			

Simsimjogak

Spring Boot backend for an emotion-based diary service

Built a backend that connects social login, diary CRUD, AI letters, summaries, and monthly keyword reports for an emotion-based diary service.

Problem

Social login, token refresh, diary data management, and external AI response generation needed to work together without breaking the user flow.

Approach

Set up authentication around Spring Security and JWT, connected MySQL and Redis to separate user data from token state, and integrated a local AI server. When external AI responses differed from the expected format, I analyzed the response structure and strengthened the parsing logic.

Result

Created a backend structure that handles authentication, persistence, AI response generation, and monthly lookup flow in one service.

- Implemented Google and Apple social login with JWT refresh flow.
- Built diary create, read, update, delete, and monthly record query APIs.
- Analyzed external AI API response structure differences and verified fixes with test cases.

Tech Stack

Spring Boot	Java	Spring Security	JPA
MySQL	Redis	JWT	

열렬히.뛰기

Go and SQLite blog for long-lived study notes

Built a public site and admin editor in a single Go binary, migrating more than 1,400 study notes from Notion and GitHub.

Problem

Equations, lists, images, and internal links could break during migration; later imports and deployments also had to preserve new writing.

Approach

Audited migration defects by type, fixed conversion rules, and codified manual classification and corrections so repeated imports converge.

Result

Made the server database authoritative, separated code deployment from database replacement, and added backups and upload guards.

- Converted Notion blocks to Markdown and migrated GitHub notes, images, and internal links.
- Shared the public renderer with previews and added inline block editing.
- Added edit conflict checks and curation rules for convergent re-imports.

Tech Stack

Go	SQLite	Markdown	Content migration
Backup			

ft_irc

Event-driven IRC server

Implemented an IRC server that handles multiple user connections and channel state, learning network server structure and event-driven I/O.

Problem

Multiple client connections, command processing, and channel state could quickly make the server architecture hard to follow.

Approach

Separated the server responsibilities early and centered the implementation around kqueue-based event monitoring for connection and channel state handling.

Result

Built a practical understanding of how event-driven servers process connections, commands, and shared state.

- Designed the server structure before expanding features.
- Studied the event detection flow of kqueue, a multiplexing-based non-blocking I/O system call.
- Learned how a server manages client connections and channel state.

Tech Stack

C++ Socket kqueue IRC

Network server

minishell

Unix shell parsing and process execution

Implemented how a shell interprets input and connects commands, learning parsing and process flow through direct implementation.

Problem

A shell has to structure input according to rules such as pipes, redirection, and environment variables instead of simply executing raw strings.

Approach

Studied how input is parsed into executable structures, then connected commands with pipe, fork, dup2, and execve.

Result

Implemented the full flow from string parsing to execution planning and process chaining.

- Learned the parsing flow that turns input into executable structures.
- Implemented pipe-based command chaining.
- Understood shell command execution and process connection.

Tech Stack

C Unix process pipe fork

dup2 execve

transcendence

Service-style web backend with authentication and runtime setup

Built a backend for a real web-service flow, covering authentication, persistence, and runtime environment setup.

Problem

The API server, login, security, database, and deployment runtime needed to work together as one integrated system.

Approach

Built APIs with Django REST Framework and integrated 42 OAuth login, JWT issuance and refresh, and email-based second-factor verification links into the authentication flow.

Result

Configured an authentication flow that considered failures and token expiration, plus a PostgreSQL and Docker Compose runtime environment.

- Built the backend with Django and Django REST Framework.
- Implemented 42 OAuth login, JWT issuance, and email-based second-factor verification links.
- Set up a development environment with PostgreSQL and Docker Compose.

Tech Stack

Django	DRF	PostgreSQL	Docker
JWT	OAuth		